



本小节内容

指针的传递

很多初学者不喜欢使用指针，觉得使用指针容易出错，其实这只是因为没有掌握指针的使用场景。经过多年的实战经验总结，**指针的使用场景通常只有两个，即传递与偏移**，读者应时刻记住只有在这两种场景下使用指针，才能准确地使用指针。多加练习之后，我们会发现指针其实很简单。首先我们来看指针的传递使用场景

1 指针的传递

我们首先来看例 1.1。在本例的主函数中，定义了整型变量 *i*，其值初始化为 10，然后通过子函数修改整型变量 *i* 的值。但是，我们发现执行语句 `printf("after change i=%d\n",i);` 后，打印的 *i* 的值仍为 10，子函数 `change` 并未改变变量 *i* 的值。下面我们通过执行程序来查看为什么会出现这种情况。

【例 1.1】指针的传递使用场景。

```
#include <stdio.h>

void change(int j)
{
    j=5;
}

int main()
{
    int i=10;
    printf("before change i=%d\n",i); //这里打断点
    change(i); //在这一步按下箭头，进入 change 函数
    printf("after change i=%d\n",i);
    return 0;
}
```

例 1.1 中代码提示位置打断点，然后调试程序，在内存视图中输入 `&i`，可以看到变量 *i* 的地址是 `0x61fe1c`。按向下键（或 F7）进入 `change` 函数，这时变量 *j* 的值的确为 10，但是 `&j` 的值为 `0x61fdf0`，也就是 *j* 和 *i* 的地址并不相同。运行 `j=5` 后，`change` 函数实际修改的是地址 `0x61fdf0` 上的值，从 10 变成了 5，接着 `change` 函数执行结束，变量 *i* 的值肯定不会发生改变，**因为变量 *i* 的地址是 `0x61fe1c` 而非 `0x61fdf0`**（由于程序每次重新编译，因此大家那里的地址和我这里是不同的，这个没关系，关键是观察 *i* 和 *j* 的地址不一样，不太会造成的同学一定不要只看这个课件，要多看几遍视频来操作）。

例 1.1 的原理图如图 1.1 所示。程序的执行过程其实就是内存的变化过程，我们需要关注的



是栈空间的变化。当 main 函数开始执行时，系统会为 main 函数开辟函数栈空间，当程序走到 int i 时，main 函数的栈空间就会为变量 i 分配 4 字节大小的空间。调用 change 函数时，系统会为 change 函数重新分配新的函数栈空间，并为形参变量 j 分配 4 字节大小的空间。在调用 change(i) 时，实际上是将 i 的值赋值给 j，我们把这种效果称为值传递（C 语言的函数调用均为值传递）。因此，当我们在 change 函数的函数栈空间内修改变量 j 的值后，change 函数执行结束，其栈空间就会释放，j 就不再存在，i 的值不会改变。

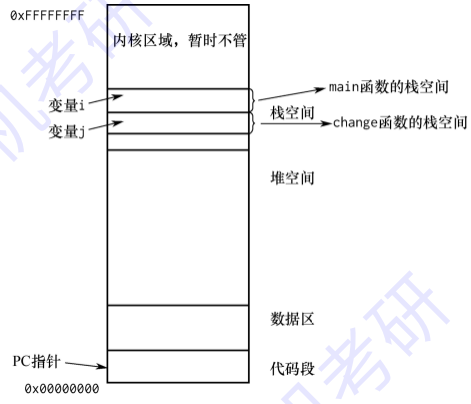


图 1.1 例 1.1 的原理图

有的读者会问，难道就不能在子函数中修改 main 函数内的某个变量的值？答案是可以的，我们将程序进行了如例 1.2 所示的修改。

【例 1.2】在子函数中修改 main 函数中某个变量的值。

```
#include <stdio.h>

void change(int* j)
{
    *j=5; //间接访问得到变量 i
}

//指针的传递
int main()
{
    int i=10;
    printf("before change i=%d\n",i);
    change(&i); //传递变量 i 的地址
    printf("after change i=%d\n",i);
    return 0;
}
```

我们可以看到程序执行后，语句 printf("after change i=%d\n",i);打印的 i 的值为 5，难道



C 语言函数调用值传递的原理变了？并非如此，我们将变量 i 的地址传递给 `change` 函数时，实际效果是 $j=&i$ ，依然是值传递，只是这时我们的 j 是一个指针变量，内部存储的是变量 i 的地址，所以通过 $*j$ 就间接访问到了与变量 i 相同的区域，通过 $*j=5$ 就实现了对变量 i 的值的改变。通过单步调试，我们依然可以看到变量 j 自身的地址是与变量 i 的地址依然不相等。